

# Next.js

## Innovation or Over-Engineering?

Is it complexity or just React evolution?

# The Golden Era

- Mental Model: **“It’s just JavaScript.”**

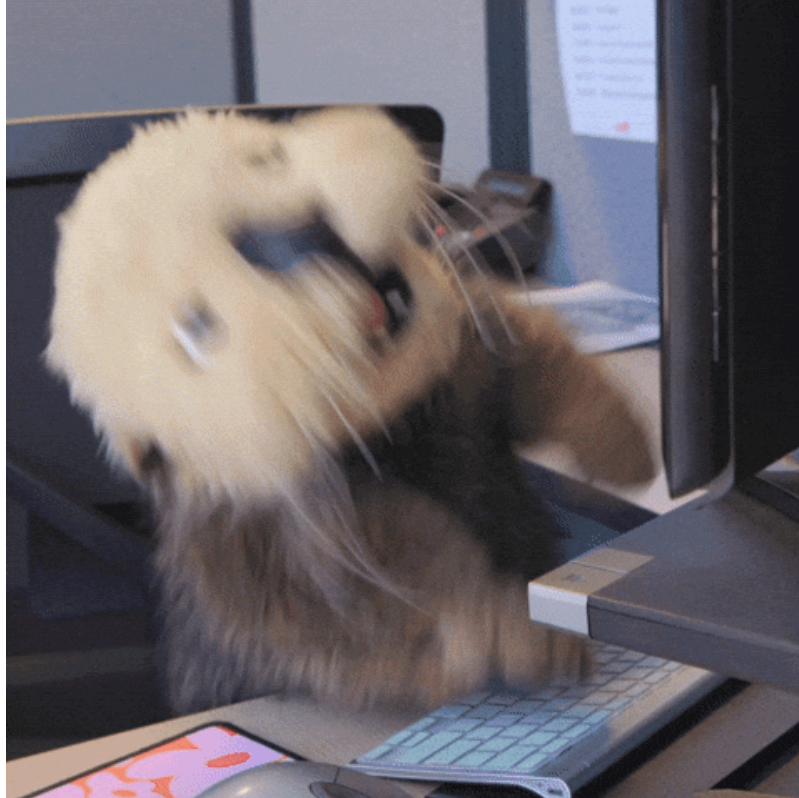
## Everywhere.

- Linters? JavaScript.
- Build tools? JavaScript.
- Compilers/Transpilers? Also JavaScript... for some reason.
- useEffects? Everywhere.

Packages to expose the character "a"? Only in JavaScript. <https://www.npmjs.com/package/@characters/a>

Tiny mental model. Predictable lifecycle. Almost zero magic.

Freeeeedom!



# With Next.js Pages Router

- Mental Model: **React with a server**
- `/pages/index.js` → Becomes your homepage. Magic, but the \*good\* kind.
- `getServerSideProps` → “I swear it’s just a function.”
- `getStaticProps` → Time-traveling data. Build once, serve forever.
- `getStaticPaths` → Because your blog definitely has 14,000 pages.

The functions must be named exactly right, or the magic breaks.

Simple conventions. ¿Minimal? surprise. ¿predictable?



## pages/index.js

```
1 export default function Home() {  
2   return <h1>Hello from 2016</h1>;  
3 }
```

## pages/blog/[slug].js

```
1 export async function getStaticPaths() {  
2   return {  
3     paths: [{ params: { slug: "hello-world" } }],  
4     fallback: false,  
5   };  
6 }  
7  
8 export async function getStaticProps({ params }) {  
9   return {  
10    props: { slug: params.slug },  
11  };  
12 }  
13  
14 export default function Blog({ slug }) {  
15   return <h2>Blog post: {slug}</h2>;  
16 }
```

# Simple, Yes... But Also...

## Data Fetching Soup

`getStaticProps` + `getStaticPaths` + `getServerSideProps`

Which one should I use?

## Bundles That Accidentally Bench-Press 40kg

Everything becomes client-side JavaScript unless you *carefully* avoid it. (*Spoiler: we did not carefully avoid it.*)

So yes, simple. But also... ¿accidentally chaotic?. Yet nextjs worked great for many use cases, but it needed to evolve.

# So... Where are we now?

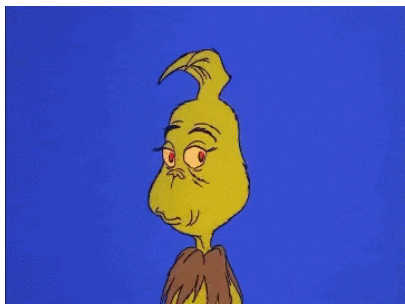
## The PhD Era.

How many PhDs in caching do I need to build a button?

- Motto: "It's just JavaScript... maybe... sometimes"
- Mental Model: Where does this code run? Only the lord knows.

Caption: Where am I running? What is cached? Who wrote this?

You want evolution so you meant over-engineering?



# With Next.js App Router

Everything is a Server Component... until it suddenly isn't.

- `app/` directory: Because we needed *one more* way to make pages.
- `page.js` : Your new god. It wraps everything. Forever.
- `loading.js` : Because your application needs a bit of suspense.
- `error.js` : Because errors need their own special place.
- `"use client"`, `"use server"` : A threat, not a directive. The beginning of the end.

App Router simplifies things... by introducing 17 new concepts.



# App Router: To Be Fair... It is Simpler (Sometimes)

File-based routing becomes *really* clean

app/dashboard/page.js

```
1  export default function Dashboard() {  
2    return <h1>Dashboard</h1>;  
3  }
```

One file = one route. Zero ceremony.

Server Components make data fetching amazingly simple

```
1  export default async function Page() {  
2    const data = await fetch("https://api.example.com").then((r) => r.json());  
3    return <pre>{JSON.stringify(data)}</pre>;  
4  }
```

No `getStaticProps`. No `getServerSideProps`. Just *async* components. Very nice, no reinvented wheel, just following the standards.

# Nested layouts without losing your sanity

app/layout.js

```
1  export default function RootLayout({ children }) {  
2    return (  
3      <html>  
4        <body>{children}</body>  
5      </html>  
6    );  
7  }
```

A layout system that doesn't need a PhD in “\_app.js gymnastics.”

So even if Next.js App Router is more complex and has a bit of more magic, it simplifies many things and improves developer experience in many ways.

We have now a clear separation between Layouts, Pages, Templates, and Components. Everything has its place. Everything makes sense. Everything is great.

Then did we did not over-engineer right? Is just regular evolution right?



# Directives - Magic or Madness?

## use cache

Allows you to mark a route, React component, or a function as cacheable.

```
1  // File level
2  "use cache";
3
4  export default async function Page() {
5    // ...
6  }
7
8  // Component level
9  export async function MyComponent() {
10    "use cache";
11    return <></>;
12  }
13
14  // Function level
15  export async function getData() {
16    "use cache";
17    const data = await fetch("/api/data");
18    return data;
19  }
```

## use cache: private

Works just like use cache, but allows you to use runtime APIs like cookies, headers, or search params.

```
1  async function getRecommendations(productId: string) {
2    "use cache: private";
3    cacheTag(`recommendations-${productId}`);
4    cacheLife({ stale: 60 });
5    ...
6  }
7
8  export default async function ProductPage({
9    params,
10   }: {
11     params: Promise<{ id: string }>,
12   }) {
13     const recommendations = await getRecommendations(productId);
14     ....
15   }
```

## use cache: remote

Lets you declaratively specify that a cached output should be stored in a remote cache instead of in-memory.

```
1  async function getProductsByCategory(category: string) {
2    'use cache: remote'
3    return db.products.findByCategory(category)
4  }
5
6  export default async function ProductsPage({
7    params,
8    searchParams,
9  }: {
10    params: Promise<{ category: string }>
11    searchParams: Promise<{ minPrice?: string }>
12  }) {
13    const products = await getProductsByCategory(category)
14    ...
15  }
```

## use client

Declares an entry point for the components to be rendered on the client side.

```
1  "use client";
2
3  import { useState } from "react";
4
5  export default function Counter() {
6    const [count, setCount] = useState(0);
7
8    return (
9      <div>
10        <p>Count: {count}</p>
11        <button onClick={() => setCount(count + 1)}>Increment</button>
12      </div>
13    );
14  }
```

## use server

Designates a function or file to be executed on the server side.

## use server at the top of a file

```
1  "use server";
2  import { db } from "@lib/db"; // Your database client
3
4  export async function fetchUsers() {
5    const users = await db.user.findMany();
6    return users;
7  }
```

## use server inline

```
1  export default async function PostPage({ params }: { params: { id: string } }) {
2    const post = await getPost(params.id);
3    async function updatePost(formData: FormData) {
4      "use server";
5      await savePost(params.id, formData);
6      revalidatePath(`/posts/${params.id}`);
7    }
8    return <EditPost updatePostAction={updatePost} post={post} />;
9  }
```

# React joins the party with Directives for the compiler

## use memo

```
1  function MyComponent() {  
2    "use memo";  
3    // ...  
4  }
```

Marks a function for React Compiler optimization.

## use no memo

```
1  function MyComponent() {  
2    "use no memo";  
3    // ...  
4  }
```

Prevents a function from being optimized by React Compiler.

# Others are also joining

TypeGPU <https://docs.swmansion.com/TypeGPU/>

`use gpu`

```
1  const neighborhood = (a: number, r: number) => {  
2    "use gpu";  
3    return d.vec2f(a - r, a + r);  
4  };
```

Allows the function to be picked up by our dedicated build plugin (unplugin-typegpu) and transformed into a format TypeGPU can understand.



# The "Simplicity" Trap

If modern Next.js feels like PHP... should we go back to PHP?

**No.** Because modern UX demands:

- Instant transitions
- Optimistic UI
- Rich interactivity
- Offline support
- Zero loading states
- Many more...

The Reality is that complexity is conserved. You either handle it in the Framework (Next.js), or you handle it in your Own ¿Spagetti? Code.

The problem isn't complexity itself; it's Accidental Complexity vs. Essential Complexity. You need both but at what degree?

# Accidental vs. Essential Complexity

Some complexity is nature. Some is... our fault.

## Essential Complexity

The real, unavoidable problems:

data fetching, routing, caching, rendering, performance.

## Accidental Complexity

The problems we created along the way:

- “Is this server or client?” — a daily existential crisis.
- `"use client"` — the scarlet letter of modern React.
- RSC boundaries that feel like zoning laws.
- Caching rules that require a flowchart and emotional support.

Next.js App Router tries to reduce essential complexity... but sometimes smuggles in a bit of accidental complexity on the side.

# When Complexity Breaks

Two major incidents in late 2025 proved that the mental model is becoming too fragile in both ways, either too much simplicity or too much complexity.

# "React2Shell" (CVE-2025-55182)

The push for Server Components blurred the line between backend and frontend, introducing a new attack vector.

## The Vulnerability:

- Mechanism: Unsafe deserialization in the React Flight protocol.
- The Attack: Attackers crafted malicious payloads sent to RSC endpoints.
- Result: Remote Code Execution (RCE) on the rendering server.

## Why This Matters

In the "Old React" (Client-side), an RCE was impossible because the code ran in the user's browser. By moving React to the server, we inherited server-side security risks, often without the security training required to handle them.

# Cloudflare Outage

A massive outage wasn't caused by hackers... **but by a React Hook.**

## The Anatomy of the Failure:

- The Code: A dashboard component with a complex useEffect.
- The Bug: A subtle dependency array mistake created an infinite re-render loop.
- The Impact: The frontend inadvertently DDoS'd Cloudflare's internal Tenant Service API.

## The Lesson

A UI bug can now take down backend infrastructure.

Frontend devs:



# So... Innovation or Over-Engineering?

## Innovation: **Yes**

- Compiler → Rust-powered, lightning-fast builds
- Streaming → React Server Components, live from the server
- Server Components → Zero-bundle magic where it counts
- 0kb client bundles → Less JS, more performance

## Over-Engineering: **Also yes**

- Dev mental model exploding → “Is this running on the server or client?”
- Server security risks → Server Components = new attack surface
- Steep learning curves → directives, RSC boundaries, caching rules

Innovation brings power... and existential dread.

Next.js now is simpler in some ways, more magical (and confusing) in others.

# Final Thought

**Complexity is a budget.** Spend it where it buys **user value**, not where it buys **developer headaches**.

## The Ferrari theory

If nextjs is a Ferrari and you only need to go to the grocery store, maybe a honda civic is a better fit.

Alternatives:

- Tanstack router
- Tanstack start

Lastly...

If Next.js becomes any more server-heavy... can we just call it **NextPHP.js**?

# Thank you

You may applaud now.